TERRAFORM-03

Objective

To execute a script, integrate Terraform with Jenkins, create a Maven job, create a parameterized Jenkins job, and learn Jenkins global variables.

GitHub Repository: https://github.com/betawins/java-Working-app.git

Task 1: Execute the Script Shown in the Video

Objective

To execute the script provided by the instructor and verify its output.

1.Using of variables

1.1 By using variable.tf file

We can execute this we want a main.tf file and we want one variable.tf file. I can use existing main.tf file.
> i will create a variable.tf
On that i will add the below content

```
variable "filename" {
default = "pets.txt"
type = string
}
variable "content" {
  default = "i love dogs"
}
```

In my main.tf file i will declare these variables

```
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "petname" {
prefix = "MR"
separator = "."
```

```
length = "1"
}
```

After that initialize the file

terraform init

After that plan the file

terraform plan

After that apply

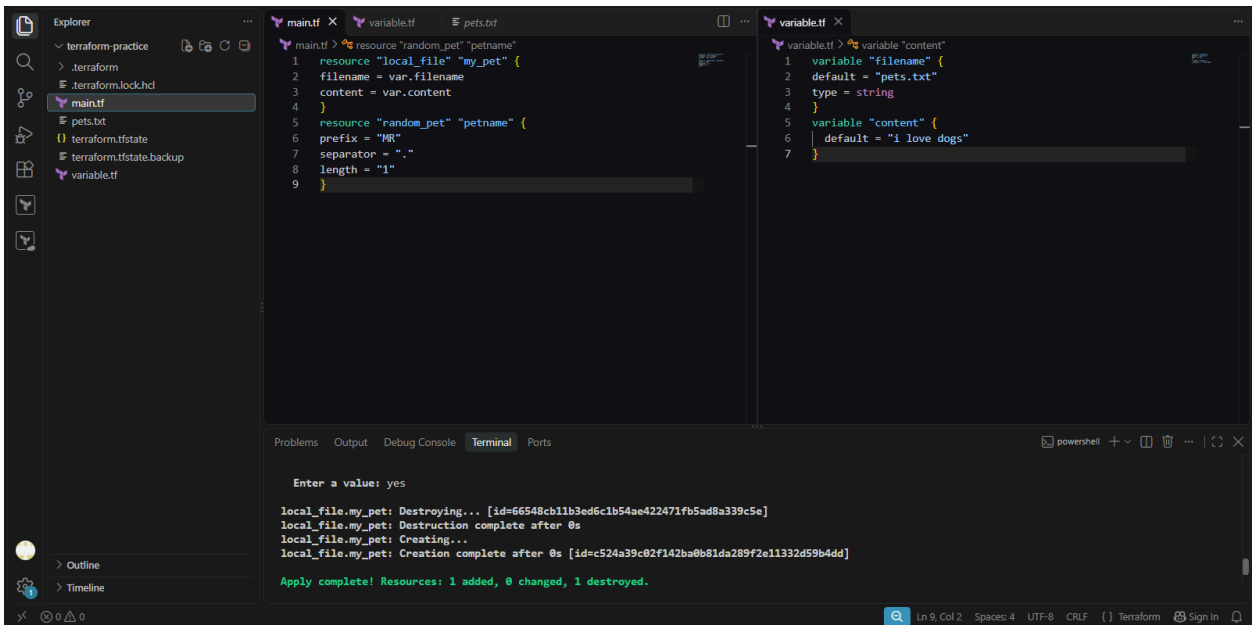
terraform apply

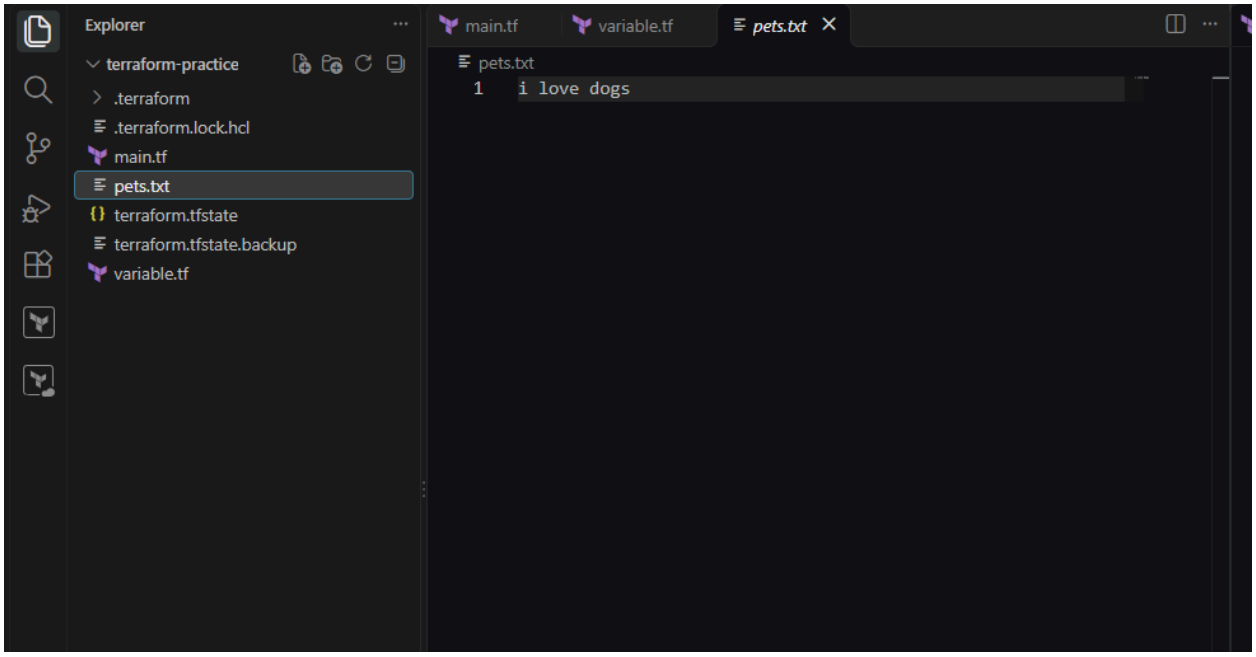
We can get the output

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

It will get the output as

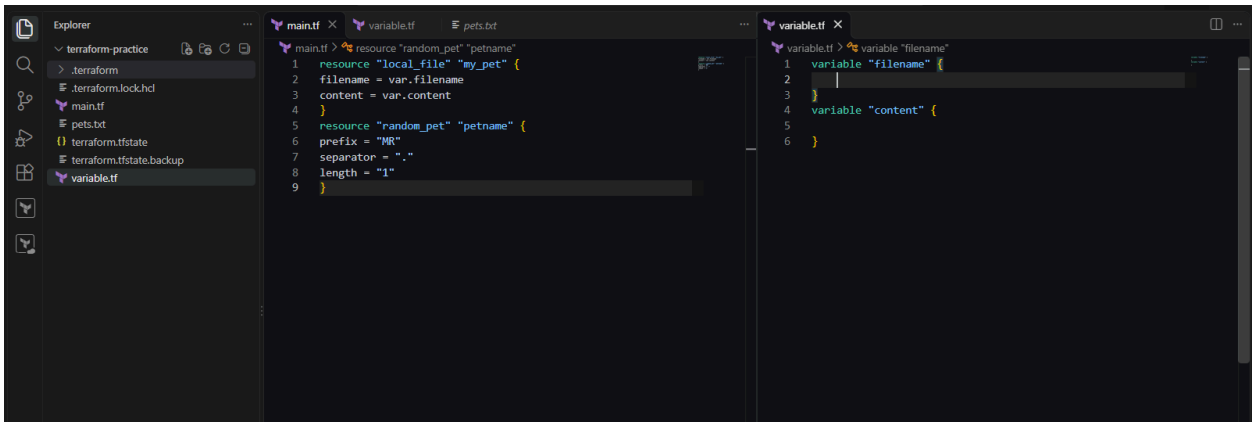
I love dogs





1.2 By using interactive mode

We can apply on runtime it will ask the values



If i can execute this it will ask the enter the values

Execute this : terraform apply

```
PS C:\terraform-practice> terraform apply
var.content
  Enter a value: i love karthik hairstyle

var.filename
  Enter a value: pets.txt

random_pet.petname: Refreshing state... [id=MR.owl]
local_file.my_pet: Refreshing state... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]
```

The the output is : i love karthik hairstyle in the pets.txt file



1.3 Command line flags

Entering the values on the apply time

The syntax is terraform apply -var “tagname=” -var “taganme=”.

Remember there is no gap between the taganems

```
PS C:\terraform-practice> terraform apply -var "filename=dog.txt" -var "content=ilovedogs"
random_pet.petname: Refreshing state... [id=MR.owl]
local_file.my_pet: Refreshing state... [id=0c088ec7968c2946300476d8c9e70725ca05e6e2]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "i love karthik hairstyle" -> "ilovedogs" # forces replacement
      ~ content_base64sha256 = "uW1D5Bb9nxIjqfNBFg+ObJIWeVEDRNNwwWkkKOwKPRc=" -> (known after apply)
      ~ content_base64sha512 = "9nhAYH2pNJEizvFS5MrrU15ch1ZiKmzpdCw/xKsvnPkYlD5c989q2Wqe3gA/VvJiAiCv1u5IR7tEOZ3FSkM7QQ==" -> (known after apply)
      ~ content_md5          = "2a246edf27c2147a043368228358b446" -> (known after apply)
      ~ content_sha1         = "0c088ec7968c2946300476d8c9e70725ca05e6e2" -> (known after apply)
      ~ content_sha256       = "b96d43e416fd9f1223a9f341160f8e6c921679510344d370c1692428ec0a3d17" -> (known after apply)
      ~ content_sha512       = "f67840607da9349122cef152e4caeb535e5c8756622a6ce9742c3fc4ab2f9cf918943e5cf7cf6ad96a9ede003f56f2620220afd6ee4847bb44399dc54
a433b41" -> (known after apply)
      ~ filename             = "pets.txt" -> "dog.txt" # forces replacement
      ~ id                   = "0c088ec7968c2946300476d8c9e70725ca05e6e2" -> (known after apply)
        # (2 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

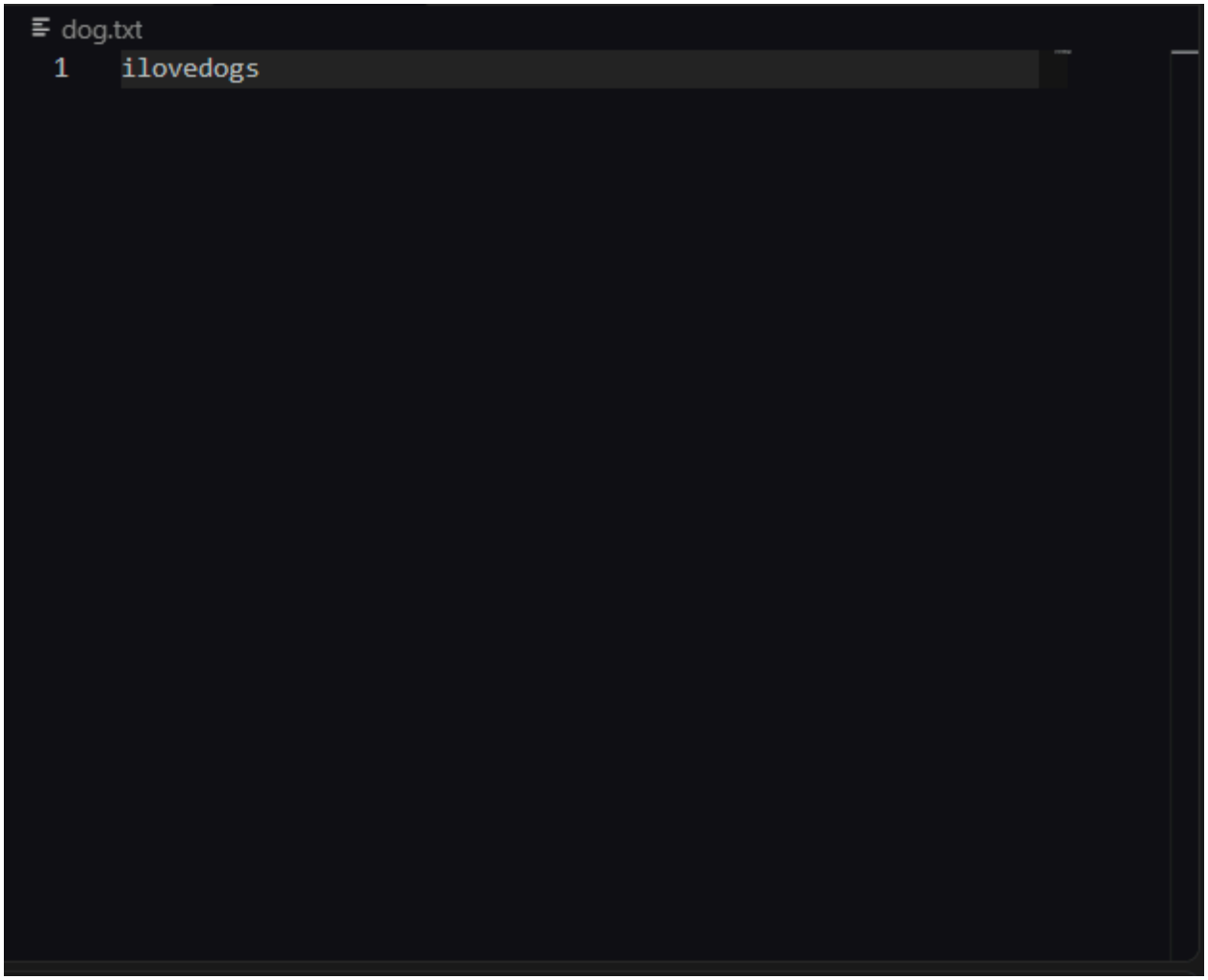
  Enter a value: yes

local_file.my_pet: Destroying... [id=0c088ec7968c2946300476d8c9e70725ca05e6e2]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=0d6dbea8f6b5a834e174fed7b07bcbb81dc3919f]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

After that we get output as

Ilovedogs in a dog.txt file



1.4 Environment variables

Syntax for linux use these

Export TF_VAR_filename = “pets.txt”

Export TF_VAR_prefix = “MR”

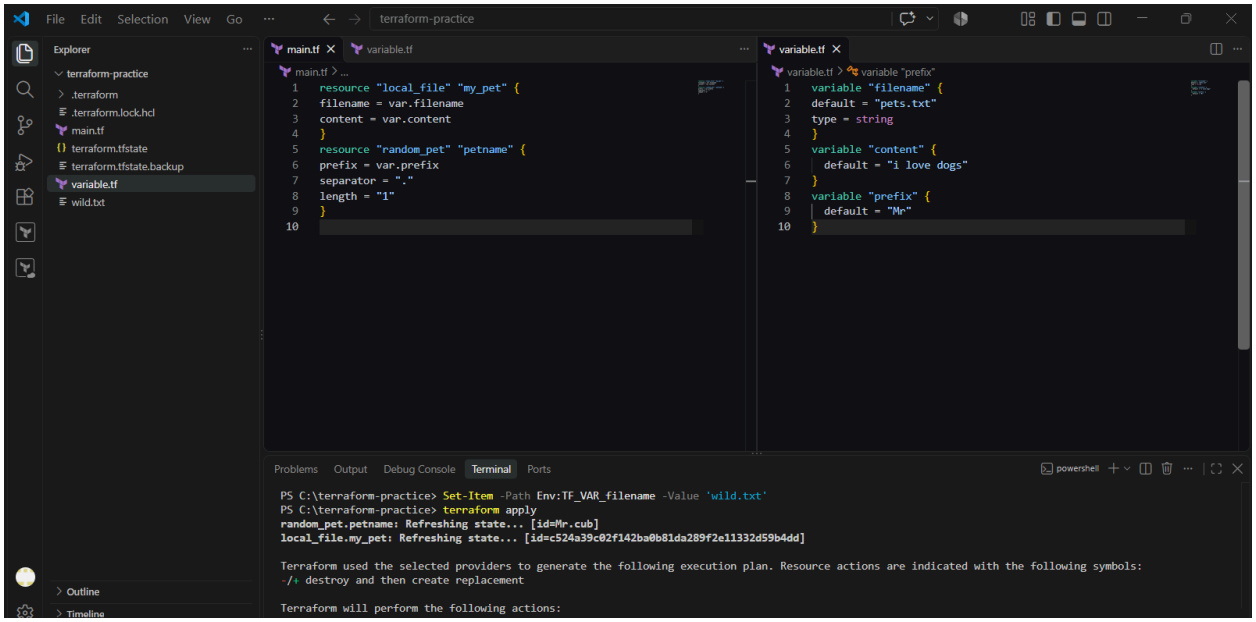
Syntax for windows use this

set_Item_path env : TF_VAR_filename -value “wild.txt”

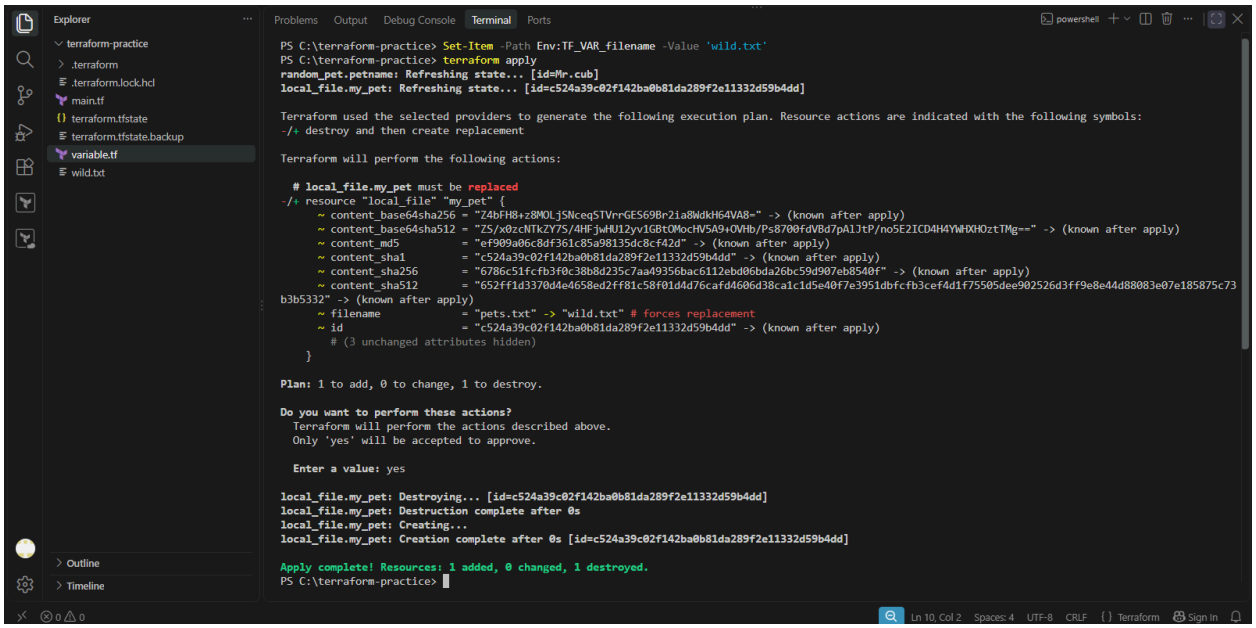
In my case i am using the windows

Set-Item -Path Env:TF_VAR_filename -Value 'wild.txt' : i can only modify the filename

If in case we can modify all.



Output is



1.5 Variable definition file

To define a variable in Terraform, assign a value using a `.tfvars` file, and execute the Terraform configuration using PowerShell in VS Code.

Step 1: Create the Terraform Project

Open the `C:\terraform-practice` folder in VS Code.

The folder contains the following files:

- `main.tf` – Main Terraform configuration.
- `variable.tf` – Variable definition file.
- `wild.txt` – Output file created by Terraform.

Step 2: Define the Variable in `variable.tf`

Open `variable.tf` and add the following code:

```
variable "filename" {
default = "pets.txt"
type = string
}
variable "content" {
  default = "i love dogs"
}
variable "prefix" {
  default = "Mr"
}
```

Step 3: Create the `variable.tfvars` File

Create a new file in the same folder and name it `variable.tfvars`.

Add the following content:

filename = "wild.txt"

Save the file.

Purpose: Provides the value `wild.txt` to the `filename` variable.

Note: `variable.tf` and `variable.tfvars` are two different files. The `.tfvars` file must exist before using it in the Terraform command.

Step 4: Execute the Terraform Command

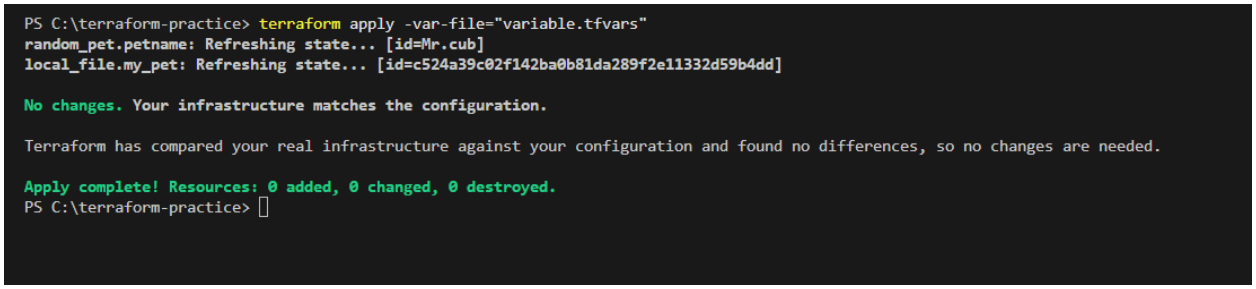
Open the VS Code PowerShell terminal and run:

terraform apply -var-file="variable.tfvars"

Purpose: Reads the variable values from `variable.tfvars` and applies the Terraform configuration.

Step 5: Confirm the Output

Terraform may display:



variable.auto.tfvar

Create a file named `variables.auto.tfvars` in the same folder.

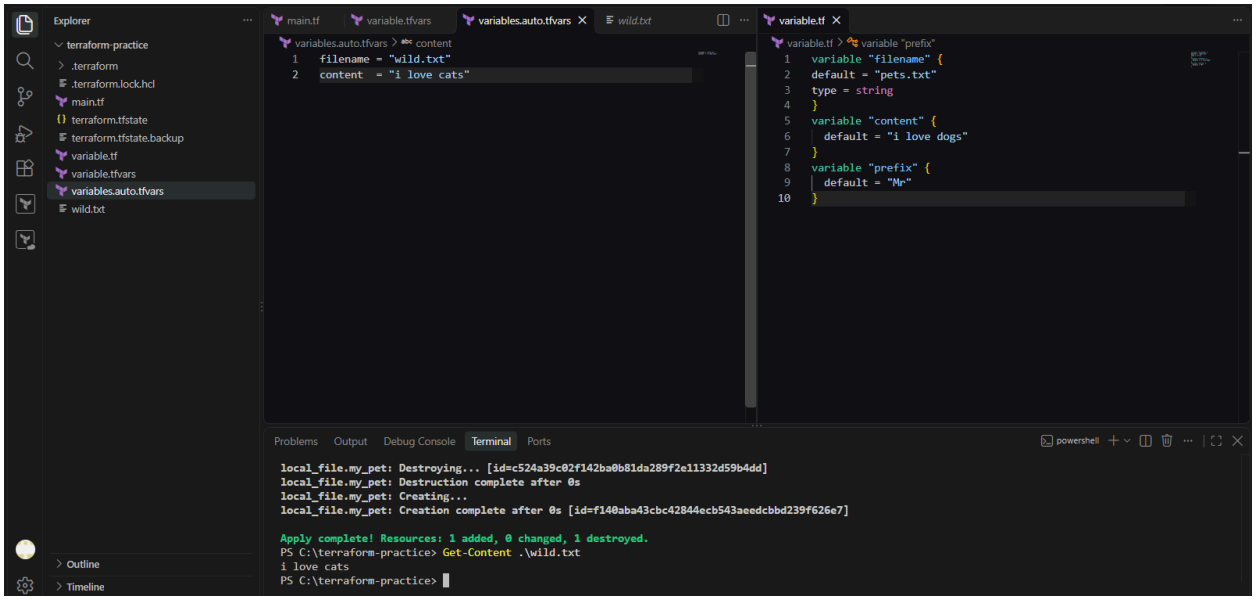
Add the following values:

filename = "wild.txt"

content = "i love cats"

Save the file using Ctrl + S.

Purpose: Supplies values that Terraform loads automatically. The values in this file override the corresponding defaults in `variable.tf`.



Run Terraform Plan

Open the VS Code PowerShell terminal and execute:

terraform plan

Purpose: Previews the changes Terraform plans to make.

Apply the Configuration

Run:

terraform apply

If Terraform asks for confirmation, type:

yes

Purpose: Applies the configuration and creates or updates the specified file.

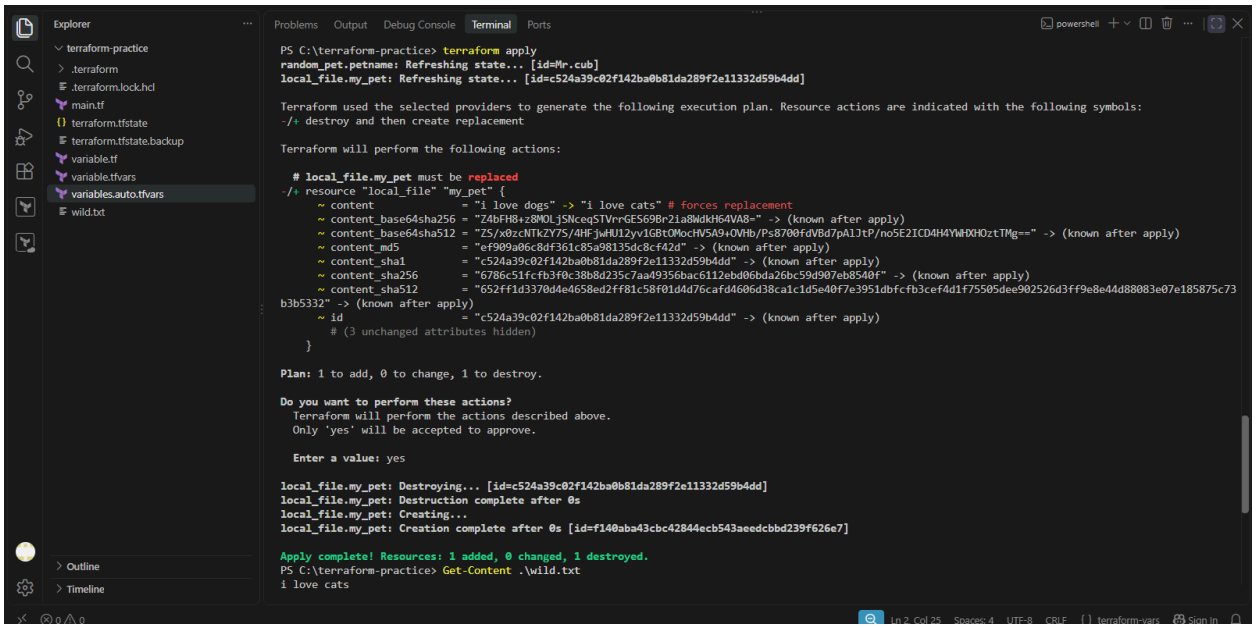
Verify the Output

Run the following command:

Get-Content .\wild.txt

Expected output:

i love cats

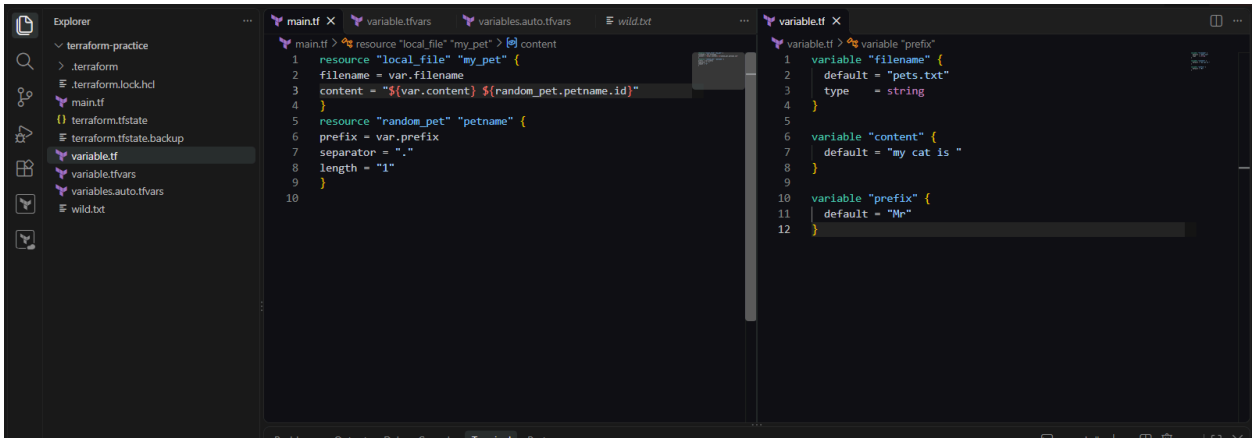


2.Resource attribute reference

If i want to link two rerouces together by using resource attributes.

```
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "petname" {
prefix = var.prefix
separator = "."
length = "1"
}
```

When we execute terraform apply it will create random id with pet name,
now i want to add this pet name in my content file (using output of one resource as input for another resource)



Execute this
Terraform init
Terraform plan
Terraform apply

```
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\terraform-practice> terraform apply
random_pet.petname: Refreshing state... [id=Mr.cub]
local_file.my_pet: Refreshing state... [id=f140aba43cbc42844ecb543aeedcbbd239f626e7]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "i love cats" -> "i love cats Mr.cub" # forces replacement
      ~ content_base64sha256 = "AUFspEf+1lx3F11vc2jU/Wv7R3Qj9RY8KcG0byJYhQs=" -> (known after apply)
      ~ content_base64sha512 = "FhjtsqYjqx60AzXS5e960fJ743YdYv0dFNQq0zaX/+8hYxoi9/2FSdM9xB3GCLl4Ysf6/5rFT4IJOpDvL46tdQ==" -> (known after apply)
      ~ content_md5          = "765ab0286886d29ac7c8dae091b071de" -> (known after apply)
      ~ content_sha1         = "f140aba43cbc42844ecb543aeedcbbd239f626e7" -> (known after apply)
      ~ content_sha256       = "01416ca447fed65c7717596f7368d4fd6bfb477423f5163c29c1b46f2258850b" -> (known after apply)
      ~ content_sha512       = "1618edb2a623ab1eb40335d2e5ef7ad1f27be3761d62fd1d14d42ad33697ffef21631a22f7fd8549d33dc41dc608b97862c7faff9ac54f82093a90ef2
f8ead75" -> (known after apply)
      ~ id                   = "f140aba43cbc42844ecb543aeedcbbd239f626e7" -> (known after apply)
        # (3 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=f140aba43cbc42844ecb543aeedcbbd239f626e7]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=221cd80a39ff30784e10af4eab19be03caa80e87]
```

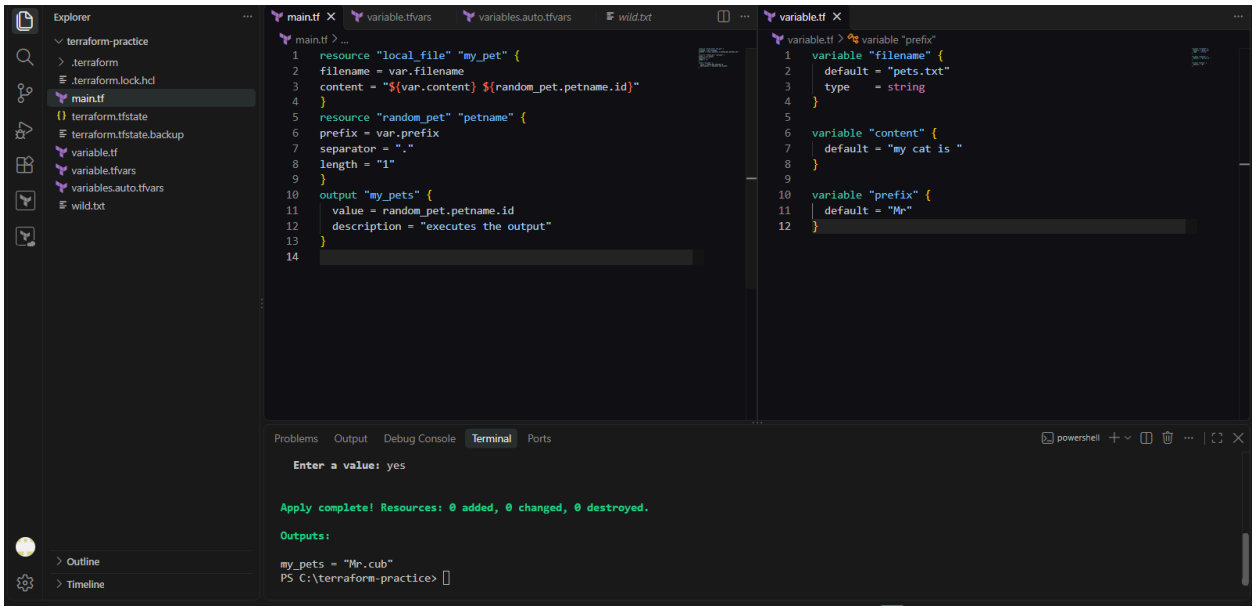
```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\terraform-practice> Get-Content .\wild.txt
i love cats Mr.cub
PS C:\terraform-practice>
```

3. Output Variables

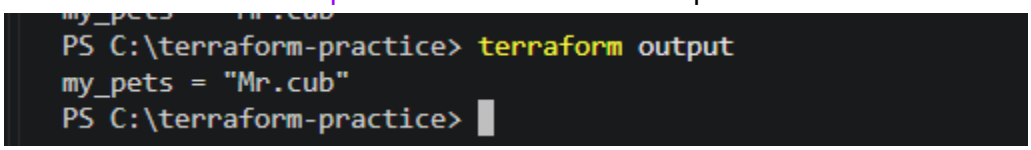
These are used to display the output of all the resources.

```
resource "local_file" "my_pet" {
filename = var.filename
content = "${var.content} ${random_pet.petname.id}"
}
resource "random_pet" "petname" {
prefix = var.prefix
separator = "."
length = "1"
}
output "my_pets" {
  value = random_pet.petname.id
  description = "executes the output"
}
```

When we use terraform apply we can see the id as output



We can use terraform output command to see the output of the resource



Observation

The script should execute successfully and produce the expected output.

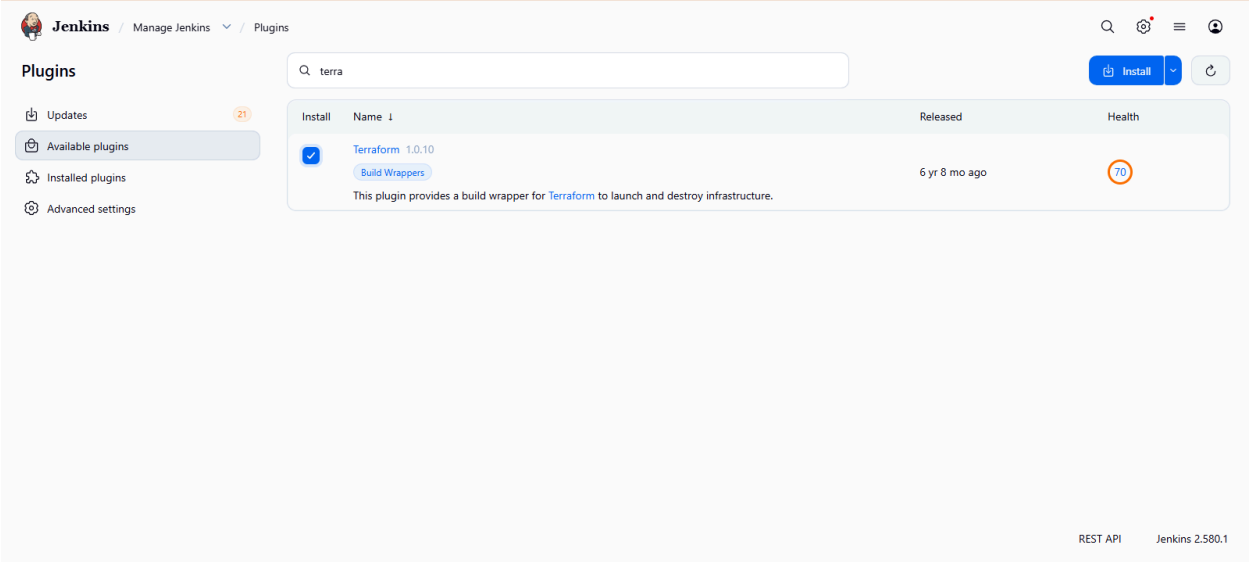
Task 2: Integrate Terraform into Jenkins Using the Terraform Plugin

Objective

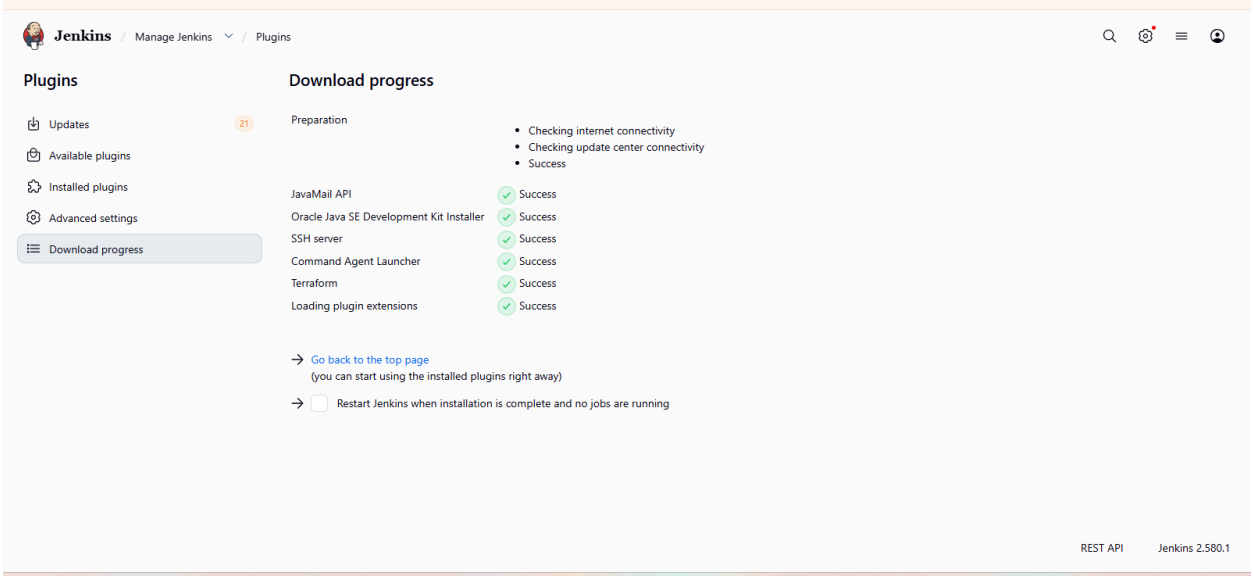
To configure Terraform in Jenkins and execute Terraform commands through a Jenkins job.

Step 1: Install the Terraform Plugin

1. Open Jenkins in your browser.
2. Click Manage Jenkins.
3. Open Plugins.
4. Search for `Terraform`.



5. Select the Terraform plugin.
6. Click Install or Install without restart, depending on the available option.



Step 2: Install Terraform

The Terraform plugin and Terraform software are different. The plugin provides Jenkins integration, while the Terraform executable runs the actual commands.

On an Jenkins server, install the required utilities:

sudo dnf update -y
sudo yum install -y wget unzip
sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
sudo yum install -y terraform

Verify the installation:

terraform -version

Expected result: The installed Terraform version should be displayed.

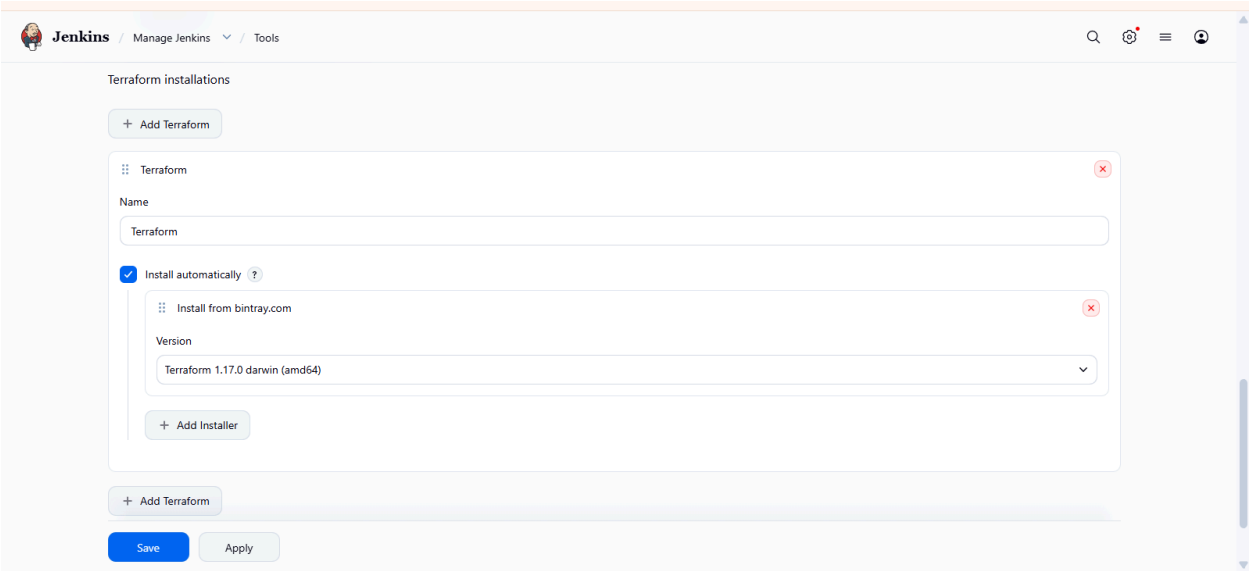
```
[root@ip-172-31-20-79 ec2-user]# terraform version
Terraform v1.17.0
on linux_amd64

[root@ip-172-31-20-79 ec2-user]#
```

Step 3: Configure Terraform in Jenkins

1. Open Manage Jenkins.

2. Click Tools or Global Tool Configuration.
3. Look for the Terraform installation section.
4. Enter the installation name as `Terraform`.
5. Configure automatic installation or the executable path, depending on the plugin options.
6. Click Save.



If the plugin does not provide a Terraform installation section, ensure Terraform is installed on the Jenkins build agent and available in its `PATH`.

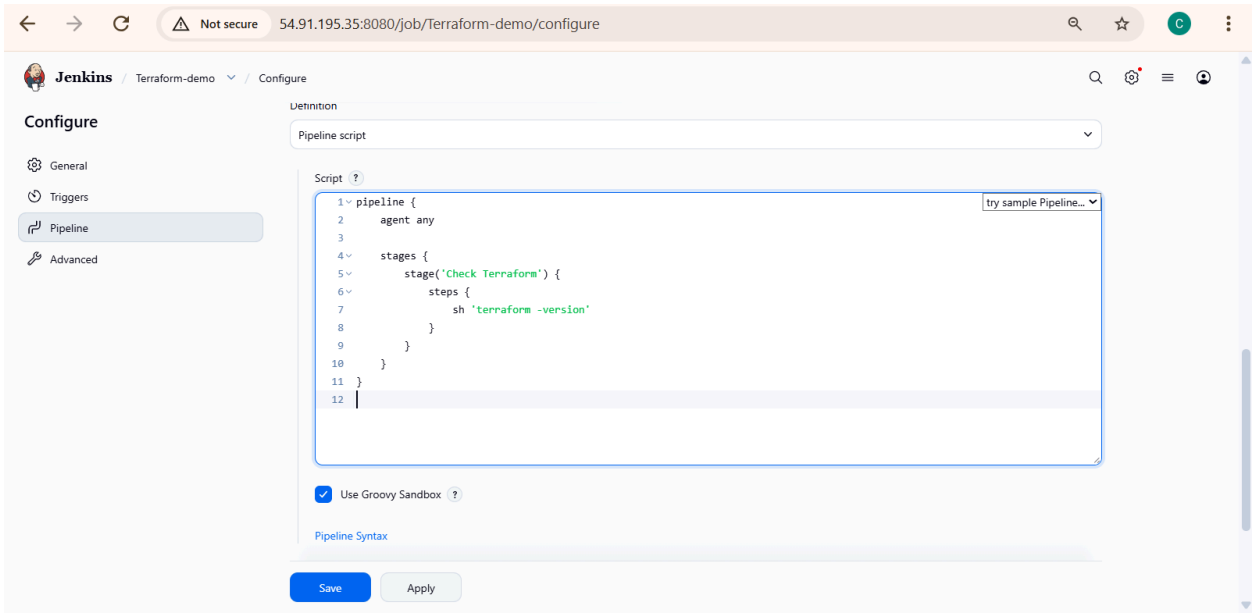
Step 4: Create a Pipeline Job

1. Click New Item.
2. Enter the job name as `terraform-demo`.
3. Select Pipeline.
4. Click OK.
5. Scroll down to the Pipeline section.
6. Select Pipeline script.
7. Enter this code:

```
pipeline {
    agent any

    stages {
        stage('Check Terraform') {
```

```
        steps {
            sh 'terraform -version'
        }
    }
  }
}
```



Step 5: Execute the Job

1. Click Save.
2. Click Build Now.
3. Open Console Output.
4. Check the Terraform version displayed in the output.



Observation

Terraform is integrated with the Jenkins job, and Jenkins can execute Terraform commands on the configured build agent.

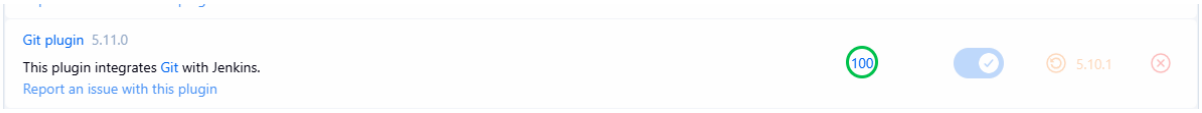
Task 3: Create a Maven Project with Git Clone and Maven Compilation

Objective

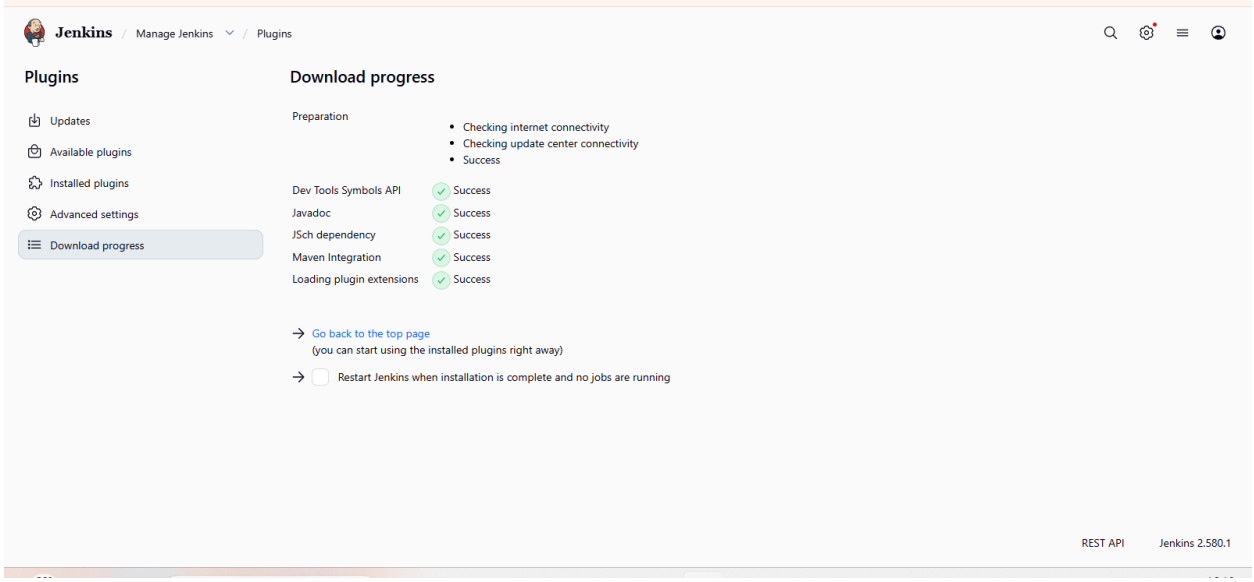
To create a Jenkins Maven Project that downloads Java code from GitHub and compiles it using Maven.

Step 1: Install Required Plugins

1. Open Manage Jenkins → Plugins.
2. Ensure the Git plugin is installed.



3. Ensure the Maven Integration plugin is installed.



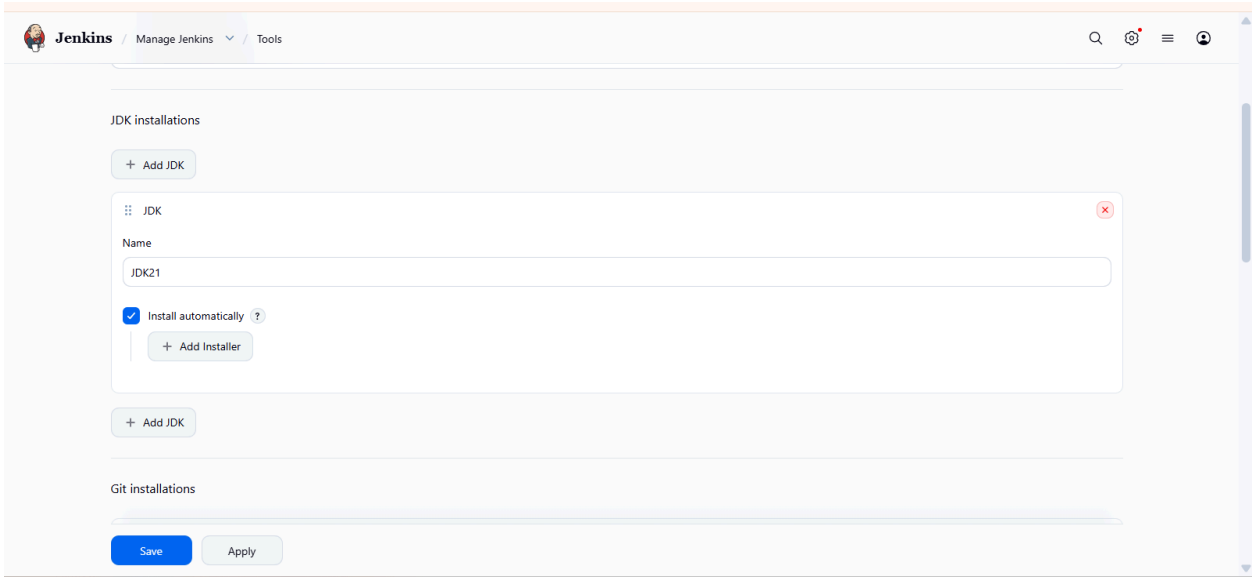
Step 2: Configure Java and Maven

1. Open Manage Jenkins → Tools.
2. Configure a JDK installation.

Enter name : JDK12

Select : install automatically

Then apply and save



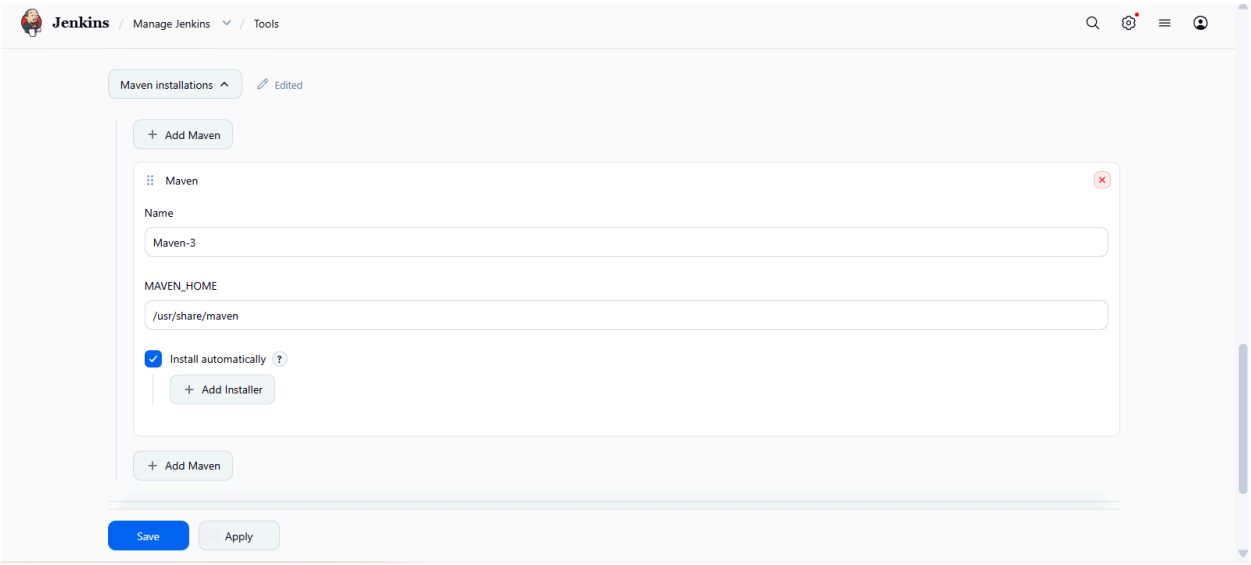
3. Configure a Maven installation.

Click Add Maven.

Enter these details:

- Name: `Maven3`
- Install automatically: Select this checkbox.
- Version: Choose an available Maven 3.x version.

Click Save.

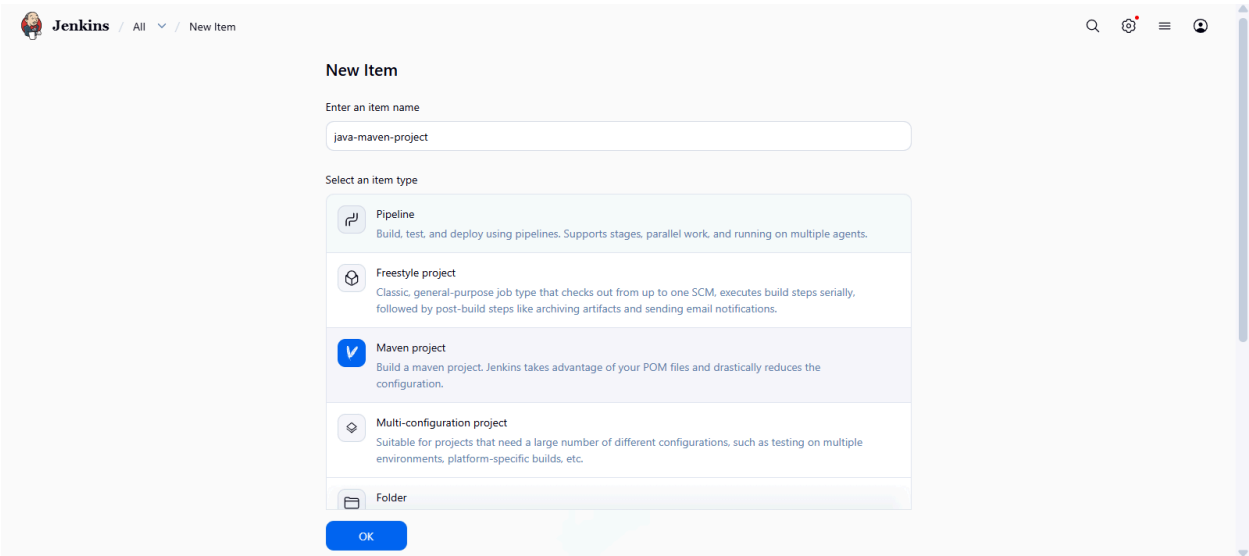


4. Save the configuration.

Jenkins needs Java and Maven to compile the application.

Step 3: Create a Maven Project

1. Click New Item.
2. Enter the job name as `java-maven-project`.
3. Select Maven project.
4. Click OK.



If you cannot see Maven Project, check whether the Maven Integration plugin is installed.

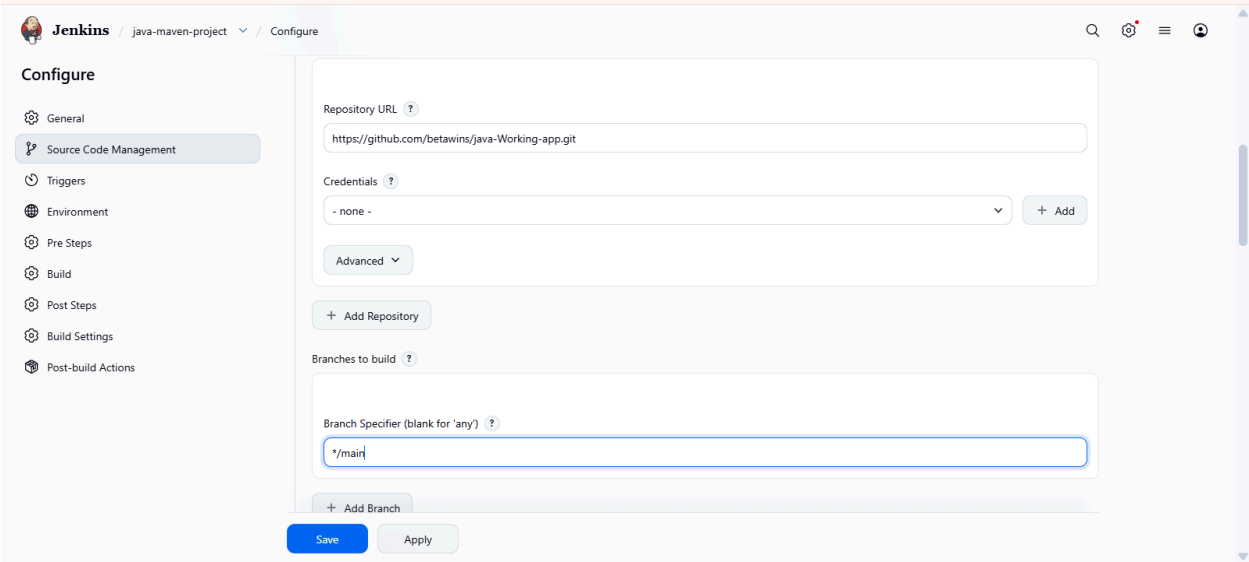
Step 4: Configure Git Clone

1. Find the Source Code Management section.
2. Select Git.
3. Enter the repository URL:

https://github.com/betawins/java-Working-app.git

4. Enter the correct branch name, such as `*/main` if the repository's branch is `main`.

This configuration allows Jenkins to download the Java source code.



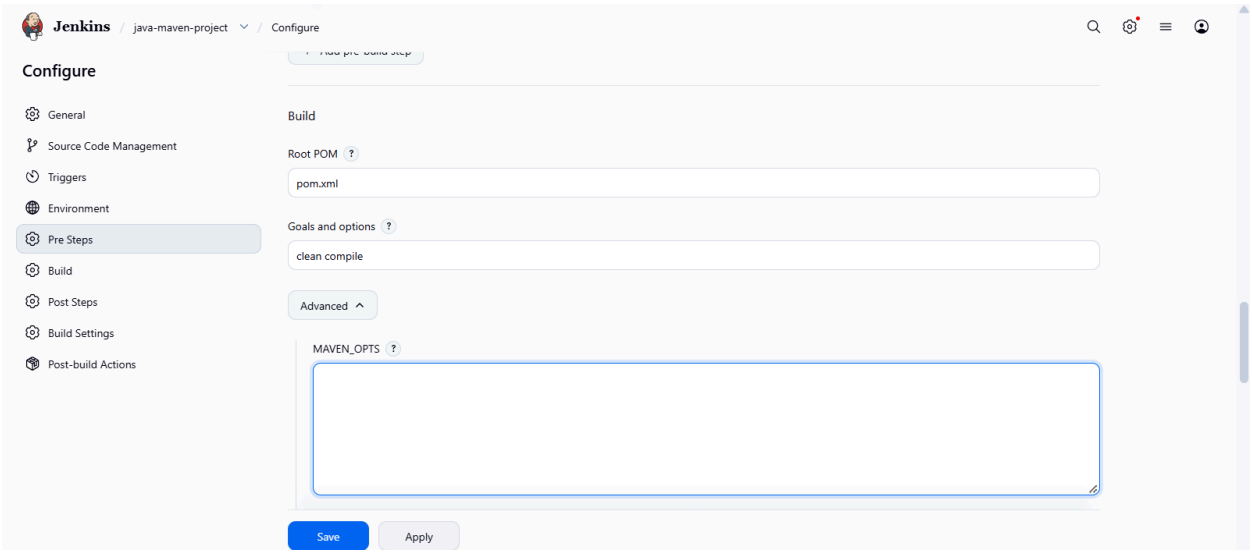
Step 5: Configure Maven Compilation

1. Find the Build section.
2. Select Invoke top-level Maven targets.
3. Choose the configured Maven version.
4. Enter the Maven goals:

clean compile

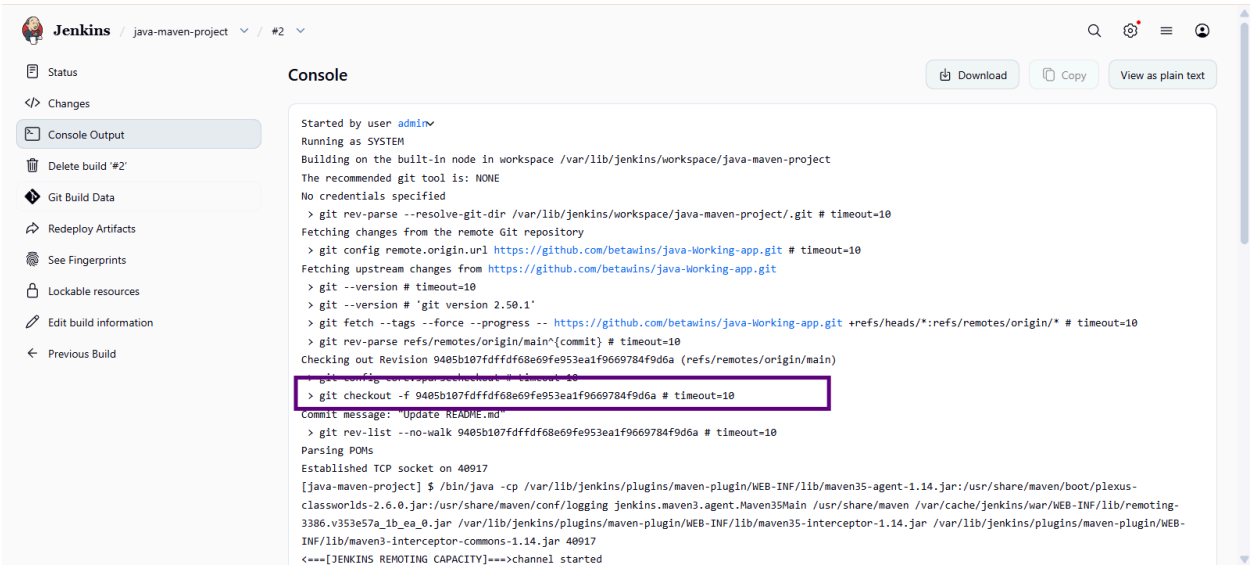
Meaning:

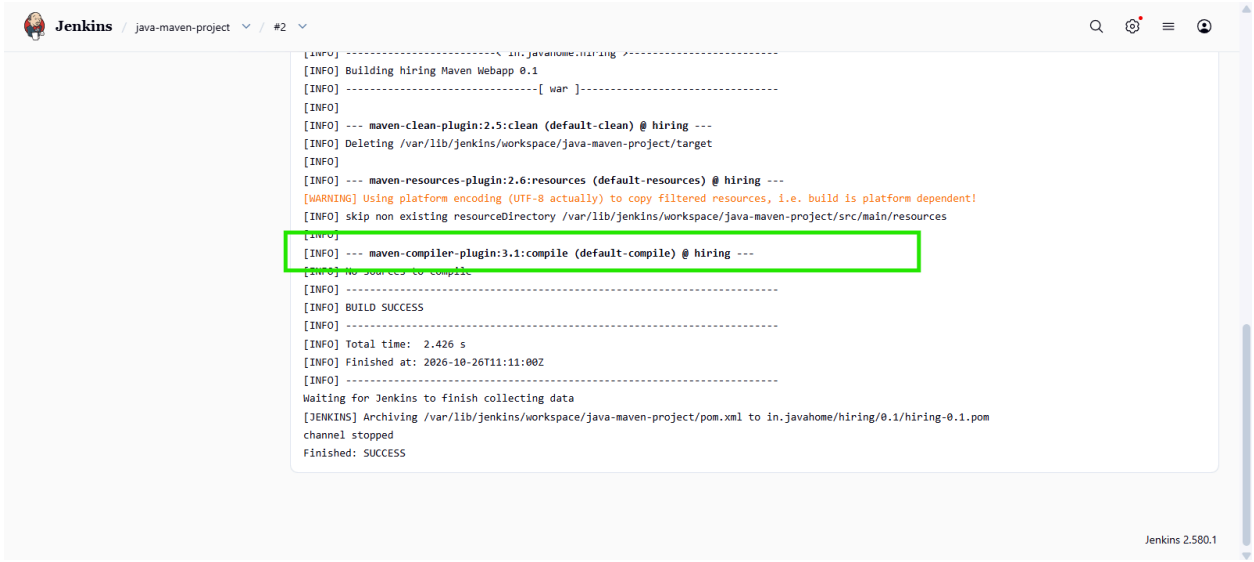
- `clean` removes previously generated build files.
- `compile` compiles the Java source code.



Step 6: Execute the Job

1. Click Save.
2. Click Build Now.
3. Open Console Output.
4. Check the Git checkout and Maven compilation messages.





Observation

Jenkins downloads the Java code from GitHub and compiles it using Maven.

Note: A traditional Maven Project performs these operations through its configuration; it does not display two custom Pipeline stages. If your instructor requires two visible stages, use the Pipeline approach in Task 4.

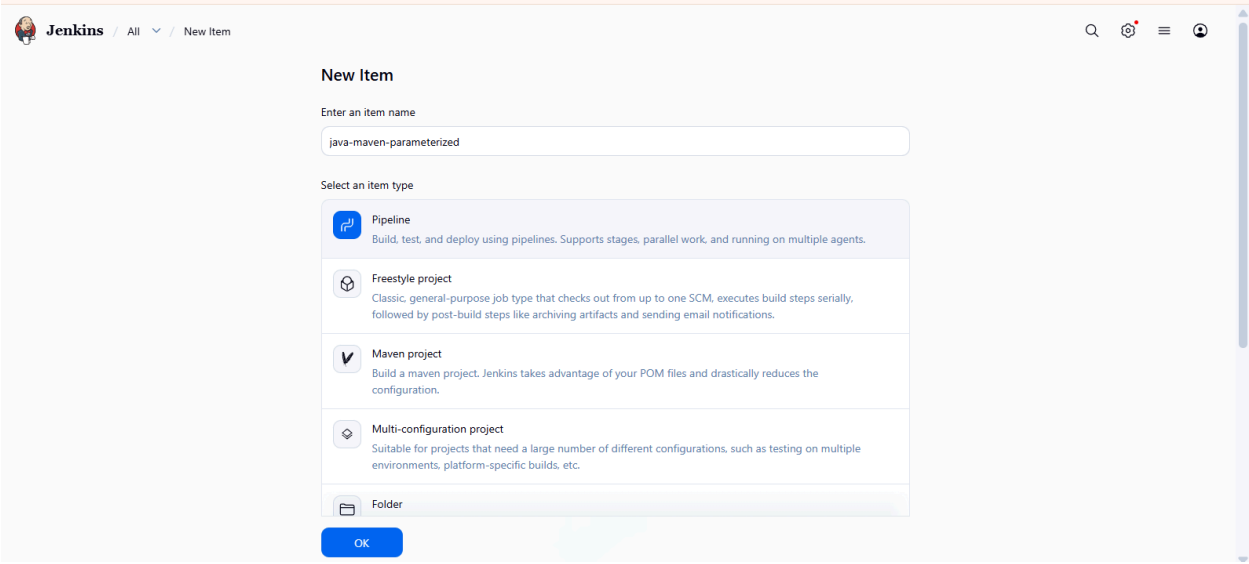
Task 4: Create a Parameterized Jenkins Job

Objective

To create a Jenkins job that accepts a parameter from the user and then clones and compiles the same Java application.

Step 1: Create the Job

1. Click New Item.
2. Enter the name `java-maven-parameterized`.
3. Select Pipeline if you need two visible stages.
4. Click OK.



Step 2: Add a Build Parameter

1. Scroll to the Pipeline job's configuration.
2. Find the parameter settings if using the job configuration UI, or define the parameter directly in the Pipeline script.
3. For the script below, you do not need to create the parameter separately in the UI.

Step 3: Configure the Pipeline

Enter the following script in the Pipeline section:

```
pipeline {
    agent any

    parameters {
        string(
            name: 'BUILD_LABEL',
            defaultValue: 'test',
            description: 'Enter a label for this build'
        )
    }

    stages {
        stage('Git Clone') {
            steps {
```

```
                git branch: 'main',
                    url: 'https://github.com/betawins/java-Working-app.git'
            }
        }

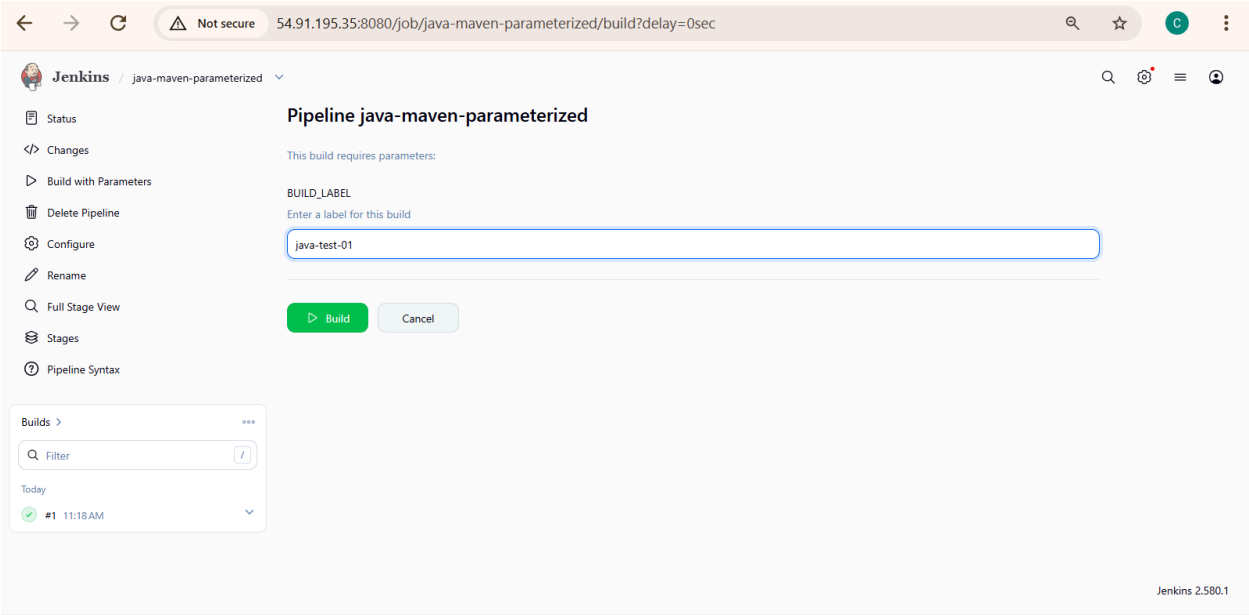
        stage('Maven Compilation') {
            steps {
                echo "Build label: ${params.BUILD_LABEL}"
                sh 'mvn clean compile'
            }
        }
    }
}
```

Before running the script:

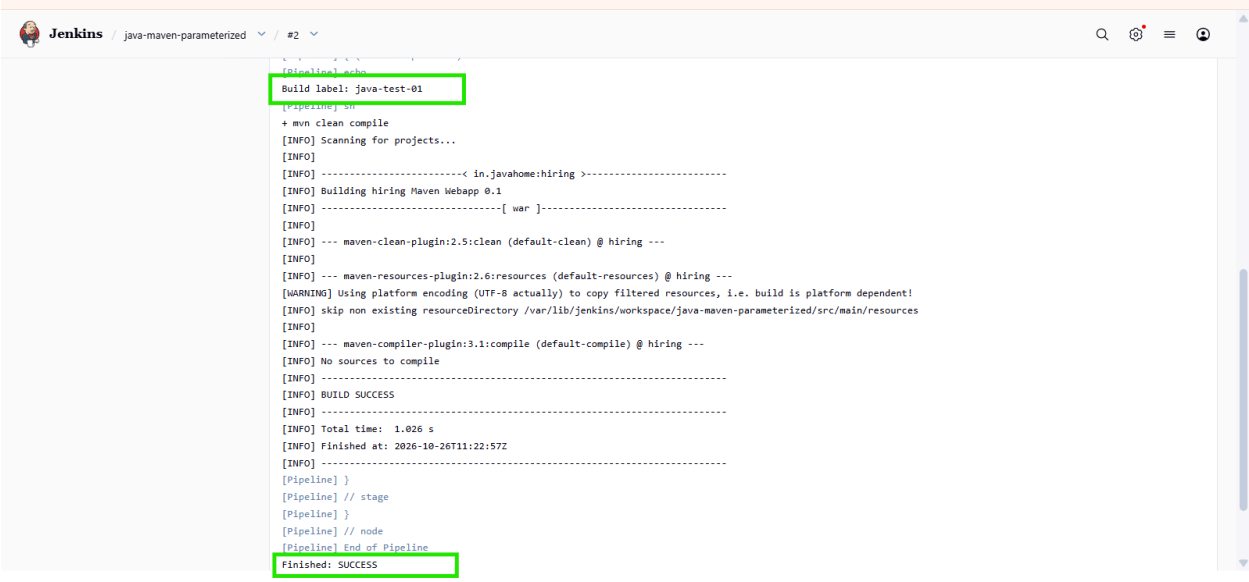
- Make sure the repository uses the `main` branch, or replace `main` with the correct branch.
- Make sure Java and Maven are installed and available on the Jenkins agent.
- If using a Windows agent, replace `sh 'mvn clean compile'` with `bat 'mvn clean compile'`.

Step 4: Save and Execute

1. Click Save.
2. Click Build with Parameters.
3. Enter a value for `BUILD_LABEL`, for example `java-test-01`.



4. Click Build.
5. Open Console Output.



Observation

The job accepts the parameter entered by the user, clones the Java repository, and compiles the code using Maven.

Difference Between Task 3 and Task 4

Task 3	Task 4
Traditional Maven Project	Pipeline job
Git checkout configured in job settings	Git checkout inside a Pipeline stage
Maven goals configured in job settings	Maven command inside a Pipeline stage
No custom Pipeline stages	Two visible stages
No parameter required	Accepts `BUILD_LABEL`

Task 5: What Are Global Variables in Jenkins?

Objective

To understand how Jenkins provides information about a job, build, workspace, and parameters.

Definition

Global variables are predefined variables and objects that Jenkins Pipelines can use to access build information and environment details.

Important Variables

Variable	Purpose
`env.JOB_NAME`	Displays the job name
`env.BUILD_NUMBER`	Displays the current build number
`env.BUILD_URL`	Displays the current build URL
`env.WORKSPACE`	Displays the workspace path
`env.NODE_NAME`	Displays the build node name
`env.BUILD_TAG`	Displays the build identifier
`params.BUILD_LABEL`	Displays the parameter entered by the user
`currentBuild.currentResult`	Displays the current build result

Jenkins provides environment variables by default, and Pipeline parameters can be accessed through `params`.

Jenkins +1

Step 1: Create a Pipeline Job

1. Click New Item.
2. Enter the name `jenkins-global-variables`.
3. Select Pipeline.
4. Click OK.

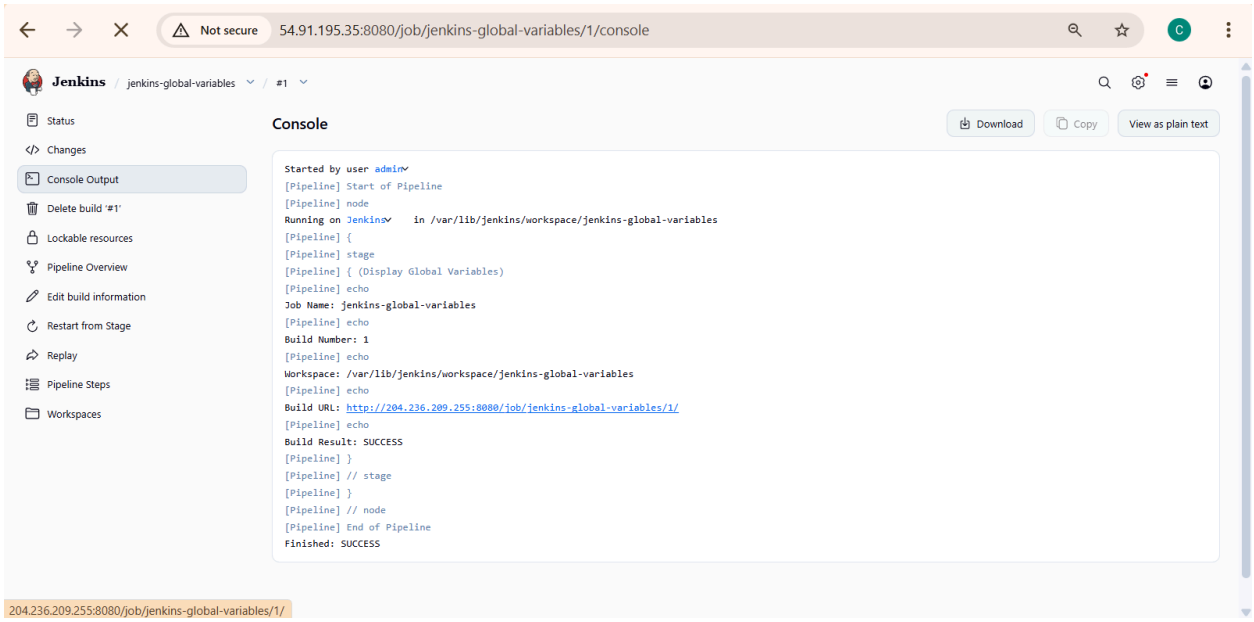
Step 2: Enter the Script

```
pipeline {
    agent any

    stages {
        stage('Display Global Variables') {
            steps {
                echo "Job Name: ${env.JOB_NAME}"
                echo "Build Number: ${env.BUILD_NUMBER}"
                echo "Workspace: ${env.WORKSPACE}"
                echo "Build URL: ${env.BUILD_URL}"
                echo "Build Result: ${currentBuild.currentResult}"
            }
        }
    }
}
```

Step 3: Execute the Job

1. Click Save.
2. Click Build Now.
3. Open Console Output.
4. Check the displayed variable values.



Observation

Jenkins displays the job name, build number, workspace path, build URL, and build status in the console output.

Interview Answer

Jenkins global variables are predefined variables and objects used to access job details, build information, environment variables, and build parameters. They help us automate CI/CD tasks in Jenkins Pipelines.

Final Result

After completing all five tasks, you should be able to:

1. Execute the script provided by your instructor.
2. Integrate Terraform with Jenkins.
3. Create a Maven job to clone and compile Java code.
4. Create a parameterized Pipeline job with Git Clone and Maven Compilation stages.
5. Display and understand Jenkins global variables.